

Chapter 8 :: Subroutines and Control Abstraction

Programming Language Pragmatics

Michael L. Scott

Review Of Stack Layout

- Allocation strategies
 - Static
 - Code
 - Globals
 - *Own* variables
 - Explicit constants (including strings, sets, other aggregates)
 - Small scalars may be stored in the instructions themselves

Review Of Stack Layout

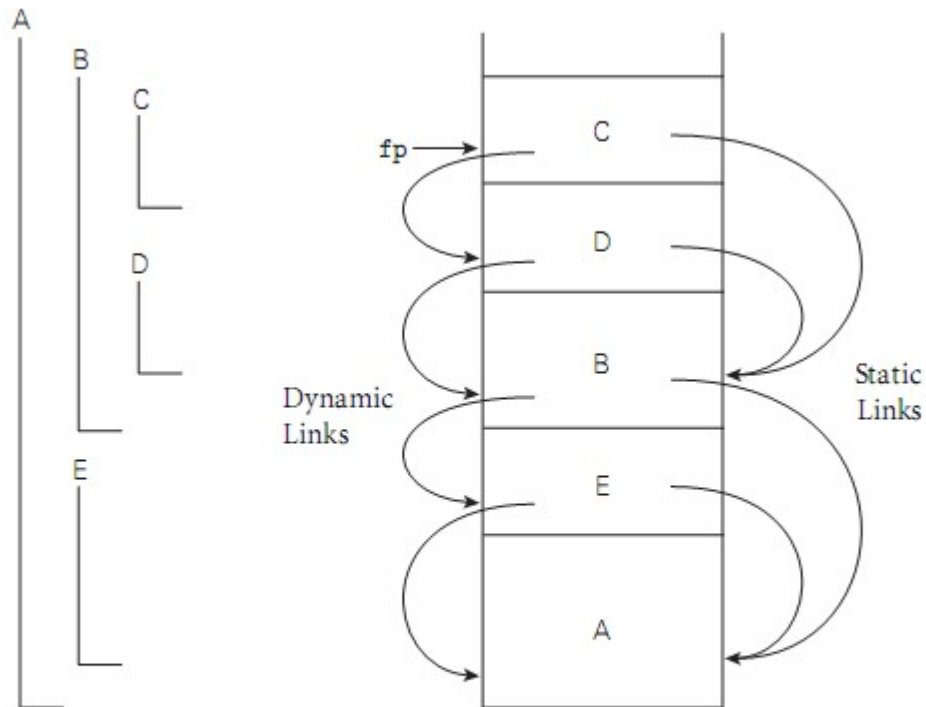


Figure 8.1 Example of subroutine nesting, taken from Figure 3.5. Within B, C, and D, all five routines are visible. Within A and E, routines A, B, and E are visible, but C and D are not. Given the calling sequence A, E, B, D, C, in that order, frames will be allocated on the stack as shown at right, with the indicated static and dynamic links.

Review Of Stack Layout

- Allocation strategies (2)
 - Stack
 - parameters
 - local variables
 - temporaries
 - bookkeeping information
 - Heap
 - dynamic allocation

Review Of Stack Layout

- Contents of a stack frame
 - bookkeeping
 - return PC (dynamic link)
 - saved registers
 - line number
 - saved display entries
 - static link
 - arguments and returns
 - local variables
 - temporaries

Calling Sequences

- Maintenance of stack is responsibility of *calling sequence* and *subroutine prolog* and *epilog* – discussed in Chapter 3
 - space is saved by putting as much in the prolog and epilog as possible
 - time *may* be saved by putting stuff in the caller instead, where more information may be known
 - e.g., there may be fewer registers IN USE at the point of call than are used SOMEWHERE in the callee

Calling Sequences

- Common strategy is to divide registers into *caller-saves* and *callee-saves* sets
 - caller uses the "callee-saves" registers first
 - "caller-saves" registers if necessary
- Local variables and arguments are assigned fixed OFFSETS from the stack pointer or frame pointer at compile time
 - some storage layouts use a separate arguments pointer
 - the VAX architecture encouraged this

Calling Sequences

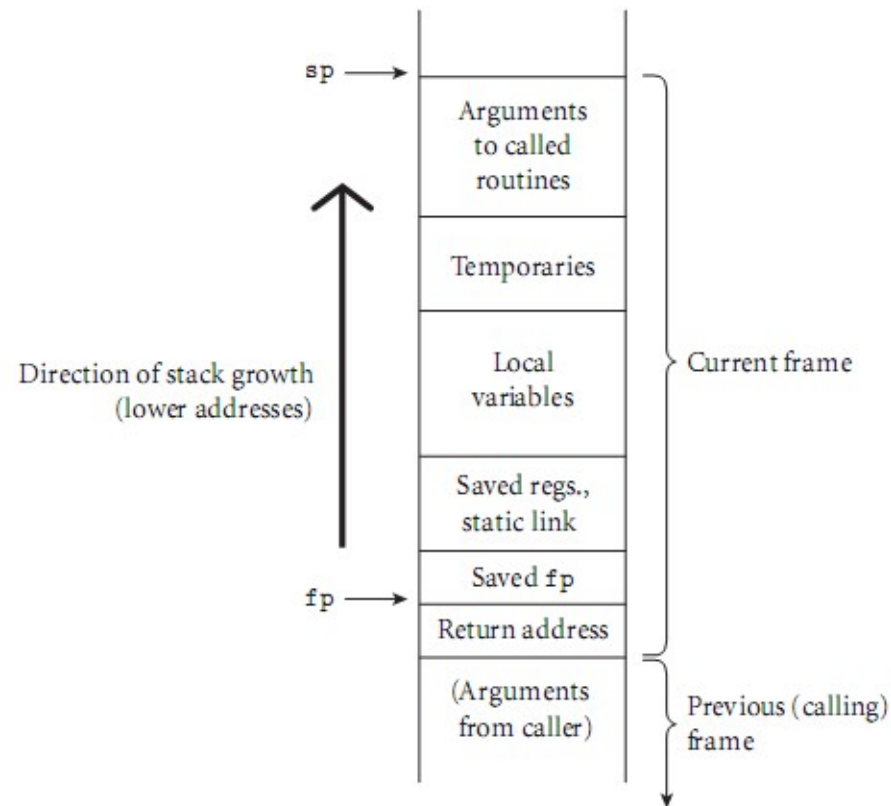


Figure 8.2 A typical stack frame. Though we draw it growing upward on the page, the stack actually grows downward toward lower addresses on most machines. Arguments are accessed at positive offsets from the fp. Local variables and temporaries are accessed at negative offsets from the fp. Arguments to be passed to called routines are assembled at the top of the frame, using positive offsets from the sp.

Calling Sequences (C on MIPS)

- Caller
 - saves into the temporaries and locals area any caller-saves registers whose values will be needed after the call
 - puts up to 4 small arguments into registers \$4-\$7 (a0-a3)
 - it depends on the types of the parameters and the order in which they appear in the argument list
 - puts the rest of the arguments into the arg build area at the top of the stack frame
 - does jal (jump and link), which puts return address into register ra and branches
 - PC+4 in \$ra

Calling Sequences (C on MIPS)

- In prolog, Callee
 - subtracts frame size from sp
 - saves callee-saves registers used anywhere inside callee
 - copies sp to fp
- In epilog, Callee
 - puts return value into registers (mem if large)
 - copies fp into sp
 - restores saved registers using sp as base
 - adds to sp to deallocate frame
 - does jra -returns control to the caller, It copies the contents of \$ra into the PC.

Calling Sequences (C on MIPS)

- After call, Caller
 - moves return value from register to wherever it's needed (if appropriate)
 - restores caller-saves registers lazily over time, as their values are needed
- All arguments have space in the stack, whether passed in registers or not
- The subroutine just begins with some of the arguments already cached in registers.

Calling Sequences (C on MIPS)

- This is a normal state of affairs; optimizing compilers keep things in registers whenever possible, flushing to memory only when they run out of registers, or when code may attempt to access the data through a pointer or from an inner scope

Calling Sequences (C on MIPS)

- Many parts of the calling sequence, prologue, and/or epilogue can be omitted in common cases
 - particularly LEAF routines (those that don't call other routines)
 - leaving things out saves time
 - simple leaf routines don't use the stack - don't even use memory – and are exceptionally fast

Special-Case Optimizations

- Many parts of the calling sequence, prologue, and epilogue can be omitted in common cases.
- If the hardware passes the return address in a register, then a leaf routine (a subroutine that makes no additional calls before returning) can simply leave it there; it does not need to save it in the stack.
- Likewise it need not save the static link or any caller-saves registers.

Special-Case Optimizations Cont..

- A subroutine with no local variables and nothing to save or restore may not even need a stack frame on a RISC machine.
- The simplest subroutines (e.g., library routines to compute the standard mathematical functions) may not touch memory at all, except to fetch instructions: they may take their arguments in registers, compute entirely in (caller-saves) registers, call no other routines, and return their results in registers.
- As a result they may be extremely fast.

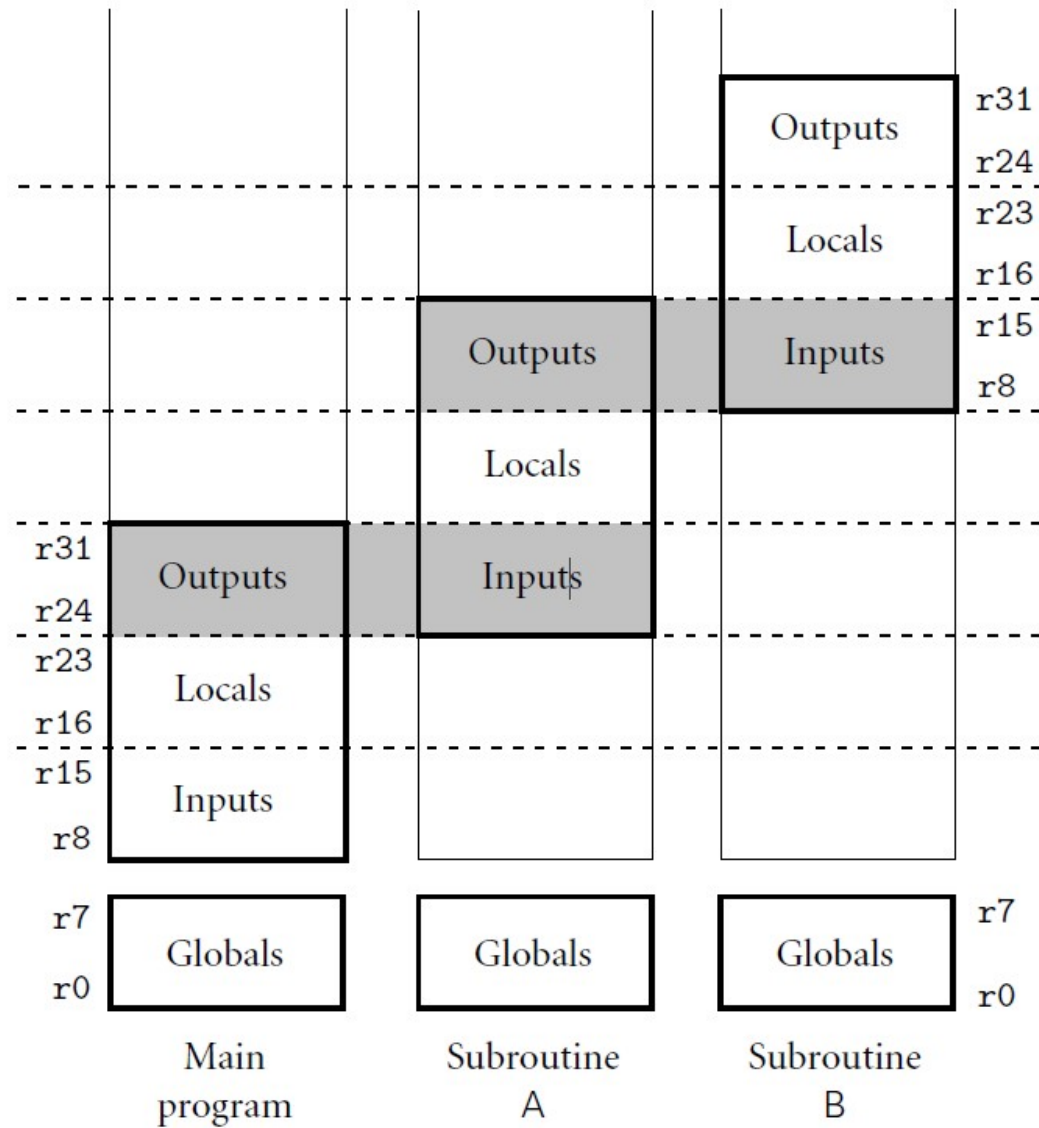
Register Windows

- As an alternative to saving and restoring registers on subroutine calls and returns, the original Berkeley RISC machines [PD80, Pat85] introduced a hardware mechanism known as register windows.
- The basic idea is to map the ISA's limited set of register names onto some subset (window) of a much larger collection of physical registers, and to change the mapping when making subroutine calls.
- Old and new mappings overlap a bit, allowing arguments to be passed (and function results returned) in the intersection.

Register Windows

- A few register names (r0–r7 in the figure) always refer to the same locations, but the rest (r8–r31 in the figure) are interpreted relative to the currently active window.
- On a subroutine call, the hardware moves to a different window.
- To facilitate the passing of parameters, the old and new windows overlap: the top few registers in the caller's window (r24–r31) are the same as the bottom few registers in the callee's window (r8–r15).

Register Windows



Register Windows

- On a machine with register windows, the compiler places values of use only within the current subroutine in the middle part of the window.
- It copies values to the upper part of the window to pass them to a called routine, within which they are read from the lower part of the window.

Register Windows

- Since the number of physical windows is fixed, a long chain of subroutine calls can cause the hardware to run off the end of the register set, resulting in a “window overflow” interrupt that drops the processor into the operating system.
- The interrupt handler then treats the set of available windows as a circular buffer.
- It copies the contents of one or more windows to memory and then resumes execution.
- Later, a “window underflow” interrupt will occur when control attempts to return into a window whose contents have been written to memory.
-
- Again the operating system recovers, by restoring the saved registers and resuming execution.
- In practice, eight windows appear to suffice to make overflow and underflow relatively rare on typical programs.